# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence** **COURSE CODE: CSA17**

---

**COURSE OUTCOME COVERED**

***CO1:*** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

*Assessment Tool Weightage: 35%*

---

**QUESTIONS: 5** **Duration : 1 Hour**
**Total Marks:50**

Annexure A

## Analytical Problem Solving

---

***Code of Conduct:***

*I **B Reddy Balu (192312499)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

Annexure A

# Analytical Problem Solving

**1.** Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

**4.** A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:

## Question 1 – Water Jug Problem

**Problem:** Two jugs (4-gallon and 3-gallon) with no markings, and a pump. Obtain exactly 2 gallons in the 4-gallon jug.

### 1. Problem formulation

- **State representation:** (x, y) where x = water in the 4-gallon jug ($0 \le x \le 4$) and y = water in the 3-gallon jug ($0 \le y \le 3$).
- **Initial state:** (0, 0) – both jugs empty.
- **Goal state:** (2, n) – exactly 2 gallons in the 4-gallon jug (n can be anything, e.g. (2, 0)).
- **Actions:** fill a jug from the pump, empty a jug on the ground, pour from one jug into the other.
- **Path cost:** 1 per action, so the best solution is the one with the fewest steps.

### 2. Production rules (state-transition rules)

| Rule | Condition | Result | Action |
|---|---|---|---|
| R1 | (x, y), $x < 4$ | (4, y) | Fill the 4-gallon jug from the pump |
| R2 | (x, y), $y < 3$ | (x, 3) | Fill the 3-gallon jug from the pump |
| R3 | (x, y), $x > 0$ | (0, y) | Empty the 4-gallon jug on the ground |
| R4 | (x, y), $y > 0$ | (x, 0) | Empty the 3-gallon jug on the ground |
| R5 | (x, y), $x + y \ge 4$, $y > 0$ | $(4, y - (4 - x))$ | Pour from 3-gal into 4-gal until the 4-gal jug is full |
| R6 | (x, y), $x + y \ge 3$, $x > 0$ | $(x - (3 - y), 3)$ | Pour from 4-gal into 3-gal until the 3-gal jug is full |
| R7 | (x, y), $x + y \le 4$, $y > 0$ | $(x + y, 0)$ | Pour all water from 3-gal into 4-gal |
| R8 | (x, y), $x + y \le 3$, $x > 0$ | $(0, x + y)$ | Pour all water from 4-gal into 3-gal |

### 3. Solution (state-space path)

| Step | 4-gal (x) | 3-gal (y) | State | Rule applied |
|---|---|---|---|---|
| 0 | 0 | 0 | (0, 0) | Initial state |
| 1 | 0 | 3 | (0, 3) | R2 – fill the 3-gallon jug |
| 2 | 3 | 0 | (3, 0) | R7 – pour 3-gal into 4-gal |
| 3 | 3 | 3 | (3, 3) | R2 – fill the 3-gallon jug again |
| 4 | 4 | 2 | (4, 2) | R5 – pour 3-gal into 4-gal until full (only 1 gallon fits) |
| 5 | 0 | 2 | (0, 2) | R3 – empty the 4-gallon jug |
| 6 | 2 | 0 | (2, 0) | R7 – pour the 2 gallons from 3-gal into 4-gal → GOAL |

**Alternative solution (also 6 steps):** (0,0) → (4,0) → (1,3) → (1,0) → (0,1) → (4,1) → (2,3), using R1, R6, R4, R8, R1, R6.

> **Answer:** The 4-gallon jug contains exactly 2 gallons after 6 operations: (0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0).

## CODE:

```
from collections import deque

def next_states(x, y):
    s = set()
    s.add((4, y))                # fill 4-gal
    s.add((x, 3))                # fill 3-gal
    s.add((0, y))                # empty 4-gal
    s.add((x, 0))                # empty 3-gal
    t = min(y, 4 - x); s.add((x + t, y - t))   # pour 3-gal -> 4-gal
    t = min(x, 3 - y); s.add((x - t, y + t))   # pour 4-gal -> 3-gal
    s.discard((x, y))
    return s
def solve():
    start = (0, 0)
    queue = deque([start])
    parent = {start: None}
    while queue:
        state = queue.popleft()
        if state[0] == 2:            # goal: 2 gallons in 4-gal jug
            path = []
            while state:
                path.append(state)
                state = parent[state]
            return path[::-1]
        for nxt in next_states(*state):
            if nxt not in parent:
                parent[nxt] = state
                queue.append(nxt)

path = solve()
for step, s in enumerate(path):
    print(f"Step {step}: {s}")
print("Total steps:", len(path) - 1)
```

## OUTPUT:

```
Step 0: (0, 0)
Step 1: (4, 0)
Step 2: (1, 3)
Step 3: (1, 0)
Step 4: (0, 1)
Step 5: (4, 1)
Step 6: (2, 3)
Total steps: 6
```

## Question 2 – Mars Rover as an Intelligent Agent (PEAS)

A Mars rover is an autonomous agent that moves over Martian terrain, collects rock/soil samples, analyses them with on-board instruments and sends the results to Earth. It is described using the PEAS framework (Performance, Environment, Actuators, Sensors).

### i. Percepts of the agent

- **Vision:** stereo/navigation cameras and mast camera images of terrain, rocks and obstacles.
- **Science sensors:** spectrometers (chemical/mineral composition), X-ray and soil-moisture sensors, microscopic imager.
- **Navigation/position:** wheel encoders (odometry), gyroscope/IMU, inclinometer (tilt), sun sensor, GPS-like localisation from landmarks.
- **Environment:** temperature, wind speed, dust level, radiation and light intensity.
- **Internal state:** battery/solar-panel power level, motor currents, wheel slip, arm and instrument status, memory left.
- **Communication:** commands and updated mission plans from Earth (via orbiter), time/sol clock.

### ii. Characterisation of the operating environment

| Property | Value | Reason |
|---|---|---|
| Observability | Partially observable | Cameras see only a limited area; hidden cracks, soft sand and underground features are not visible. |
| Agents | Single agent | Only one rover acts; other rovers/orbiters are cooperative, not competitive. |
| Determinism | Stochastic | Wheel slip, sandy slopes, dust storms and sensor noise make outcomes uncertain. |
| Episodic / Sequential | Sequential | Present moves (power use, position, damage) affect all later decisions. |
| Static / Dynamic | Dynamic | Weather, light and dust change while the rover is deliberating. |
| Discrete / Continuous | Continuous | Positions, speeds, temperatures and arm angles take continuous values. |
| Known / Unknown | Largely unknown | Terrain is only partly mapped; the rover must learn about it as it explores. |

### iii. Actions the agent can take (actuators)

- Move forward / backward, turn left / right, stop, climb or steer around obstacles, choose a safe path.
- Deploy the robotic arm, scoop soil, drill or grind rock, place instruments on a target.
- Capture images, run spectrometer/chemical analysis, store or discard data.
- Transmit data to the orbiter/Earth, receive commands.
- Manage energy: tilt/clean solar panels, recharge, heat instruments, sleep or enter “safe mode” during storms or faults.

### iv. Performance measure

- **Science return:** number and quality (scientific value) of samples correctly analysed and transmitted.
- **Coverage:** area explored and number of targets of interest reached.
- **Safety:** no collisions, falls, or getting stuck; instruments not damaged.
- **Efficiency:** energy, time and communication bandwidth used per sample.
- **Longevity:** mission lifetime of the rover (sols operated).

*Example: Performance = Σ (value of samples analysed) + area explored − (energy used + damage penalty).*

### v. Most suitable agent architecture

> **Answer: A goal-based + utility-based, model-based learning agent in a layered (hybrid) architecture.**

- **Model-based:** the environment is partially observable, so the rover must keep an internal map/state (position, terrain, power, instrument status).
- **Goal-based:** it has explicit goals ("reach rock X", "drill and analyse") and must plan sequences of actions to achieve them.
- **Utility-based:** goals conflict (science value vs. risk vs. battery), so it needs a utility function to choose, e.g. a safer path versus a more interesting but riskier rock.
- **Learning element:** improves terrain classification and traversal predictions from experience.
- **Reflex layer:** simple condition–action rules (e.g. "tilt too high → stop", "low battery → safe mode") for instant safety reactions.
- **Autonomy:** radio delay between Earth and Mars is about 4–24 minutes, so the rover cannot be remote-controlled in real time and must decide on its own.

A pure simple-reflex agent is insufficient (no memory, no planning), so the layered goal/utility architecture is chosen.

## CODE:

```
def rover_agent(percept):
    if percept["battery"] < 20:
        return "Enter safe mode and recharge"
    if percept["dust_storm"]:
        return "Stop and wait for the storm to pass"
    if percept["tilt"] > 30:
        return "Stop and re-plan a safer path"
    if percept["obstacle"]:
        return "Turn and drive around the obstacle"
    if percept["rock_of_interest"]:
        return "Deploy arm, drill and analyse the sample"
    if percept["data_ready"]:
        return "Transmit data to Earth"
    return "Move forward and explore"
```

```
percepts = [
    {"battery": 80, "dust_storm": False, "tilt": 5,  "obstacle": False, "rock_of_interest": False,
"data_ready": False},
    {"battery": 75, "dust_storm": False, "tilt": 8,  "obstacle": True,  "rock_of_interest": False,
"data_ready": False},
    {"battery": 70, "dust_storm": False, "tilt": 6,  "obstacle": False, "rock_of_interest": True,
"data_ready": False},
    {"battery": 65, "dust_storm": False, "tilt": 4,  "obstacle": False, "rock_of_interest": False,
"data_ready": True},
    {"battery": 60, "dust_storm": False, "tilt": 35, "obstacle": False, "rock_of_interest": False,
"data_ready": False},
    {"battery": 55, "dust_storm": True,  "tilt": 3,  "obstacle": False, "rock_of_interest": False,
"data_ready": False},
    {"battery": 15, "dust_storm": False, "tilt": 3,  "obstacle": False, "rock_of_interest": False,
"data_ready": False},
]
for n, p in enumerate(percepts, 1):
    print(f"Percept {n}: {p}")
    print("   Action ->", rover_agent(p))
```

## OUTPUT:

```
Percept 1: {'battery': 80, 'dust_storm': False, 'tilt': 5, 'obstacle': False, 'rock_of_interest': False, 'data_ready': False}
   Action -> Move forward and explore
Percept 2: {'battery': 75, 'dust_storm': False, 'tilt': 8, 'obstacle': True, 'rock_of_interest': False, 'data_ready': False}
   Action -> Turn and drive around the obstacle
Percept 3: {'battery': 70, 'dust_storm': False, 'tilt': 6, 'obstacle': False, 'rock_of_interest': True, 'data_ready': False}
   Action -> Deploy arm, drill and analyse the sample
Percept 4: {'battery': 65, 'dust_storm': False, 'tilt': 4, 'obstacle': False, 'rock_of_interest': False, 'data_ready': True}
   Action -> Transmit data to Earth
Percept 5: {'battery': 60, 'dust_storm': False, 'tilt': 35, 'obstacle': False, 'rock_of_interest': False, 'data_ready': False}
   Action -> Stop and re-plan a safer path
Percept 6: {'battery': 55, 'dust_storm': True, 'tilt': 3, 'obstacle': False, 'rock_of_interest': False, 'data_ready': False}
   Action -> Stop and wait for the storm to pass
Percept 7: {'battery': 15, 'dust_storm': False, 'tilt': 3, 'obstacle': False, 'rock_of_interest': False, 'data_ready': False}
   Action -> Enter safe mode and recharge
```

## Question 3 – 8-Queens Problem Formulation and Search

**Problem:** Place 8 queens on an 8×8 chessboard so that no queen attacks another (no two share a row, column or diagonal). The configuration in the given figure is not a solution because the queen in the first column and the queen in the last column are on the same diagonal.

### 1. Problem components

Two formulations are possible. The incremental formulation is better for search.

| Component | Naïve incremental | Improved incremental (preferred) |
|---|---|---|
| States | Any arrangement of 0–8 queens on the board | Any arrangement of k queens ($0 \le k \le 8$), one per column in the leftmost k columns, with no queen attacking another |
| Initial state | Empty board (no queens) | Empty board (no queens) |
| Actions | Add a queen to any empty square | Add a queen to a square in the leftmost empty column that is not attacked by any queen already placed |
| Transition model | Returns the board with the new queen added | Returns the board with the new queen added in column k+1 |
| Goal test | 8 queens on the board, none attacked | 8 queens on the board (non-attacking is guaranteed by the actions) |
| Path cost | Not relevant – only the final configuration matters (each step cost 1) | Same |
| State-space size | $64 \times 63 \times \ldots \times 57 \approx 1.8 \times 10^{14}$ | Only 2,057 states |

**Complete-state formulation (alternative):** start with all 8 queens on the board (one per column) and move a queen within its column to reduce attacks – used by local search such as hill climbing / min-conflicts.

### 2. Exploring the search space

- **Breadth-First Search (BFS):** expands all placements level by level. It finds a solution at depth 8 but stores a huge number of nodes, so memory is a problem.
- **Depth-First Search (DFS):** every solution lies at the same depth (8), so DFS is ideal – it needs only O(bm) memory. DFS with the improved formulation is simply backtracking.
- **Backtracking:** place a queen column by column; if no safe row exists in a column, return to the previous column and try its next row. This prunes the search heavily.
- **Why not uninformed search on the naïve formulation?** The $10^{14}$ states make it impractical; reducing the state space (one queen per column, attack check) is the key.

**Backtracking pseudocode:**

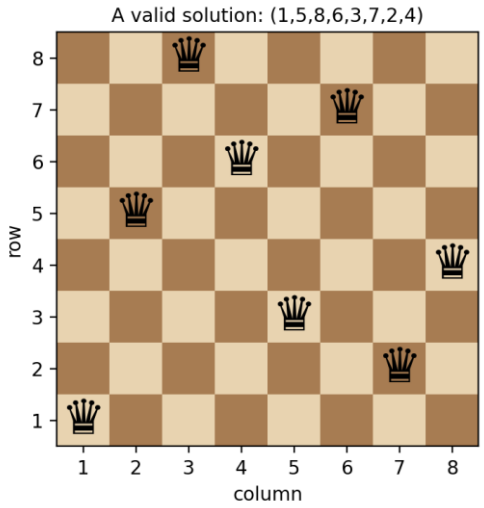
```
function PLACE-QUEENS(col, board):
  if col = 8 + 1:          # all 8 columns filled
    return board           # goal reached
  for row = 1 to 8:
```

```
    if SAFE(board, row, col):      # no same row, no same diagonal
      board[col] <- row
      result <- PLACE-QUEENS(col + 1, board)
      if result != failure: return result
      board[col] <- 0            # backtrack
  return failure

function SAFE(board, row, col):
  for c = 1 to col - 1:
    if board[c] = row or |board[c] - row| = col - c:
      return false
  return true
```

### 3. Why the given configuration fails and a valid solution

For two queens at (row r1, column c1) and (row r2, column c2), they attack diagonally if $|r1 - r2| = |c1 - c2|$. In the given figure the queens in column 1 and column 8 satisfy this condition, so the goal test fails and the search must backtrack (move one of these queens). Continuing backtracking search gives, for example, the valid solution below (row number of the queen in each column 1–8):




**Answer:** Solution (rows for columns 1–8): 1, 5, 8, 6, 3, 7, 2, 4. No two queens share a row, column or diagonal. The 8-queens problem has 92 solutions (12 distinct up to symmetry).

## CODE:

```
N = 8

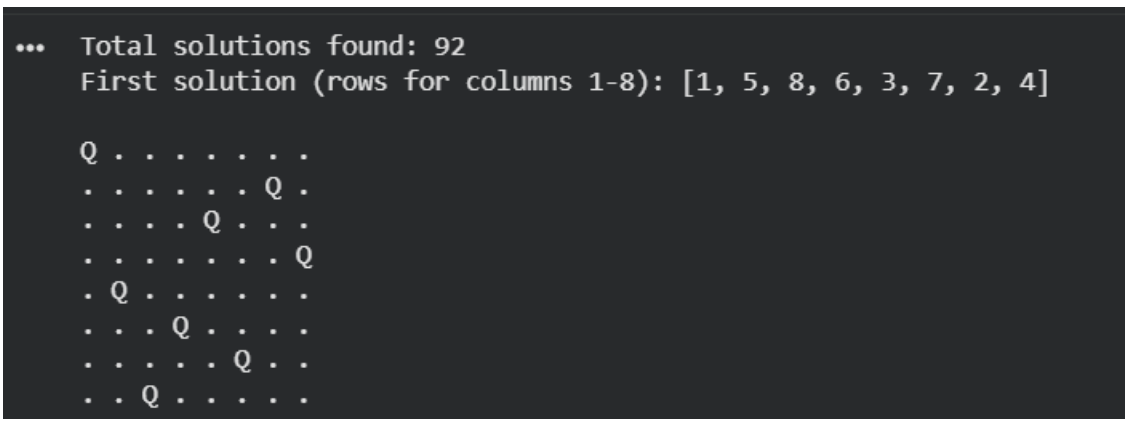
def is_safe(board, row, col):
    for c in range(col):
        if board[c] == row or abs(board[c] - row) == col - c:
            return False
    return True

def solve(board, col, solutions):
    if col == N:
        solutions.append(board[:])
        return
    for row in range(N):
        if is_safe(board, row, col):
            board[col] = row
            solve(board, col + 1, solutions)
            board[col] = -1           # backtrack

solutions = []
solve([-1] * N, 0, solutions)

print("Total solutions found:", len(solutions))
first = solutions[0]
print("First solution (rows for columns 1-8):", [r + 1 for r in first])
print()
for r in range(N):
    print(" ".join("Q" if first[c] == r else "." for c in range(N)))
```

## OUTPUT:



# Question 4 – OLA Cab Booking: Problem-Solving Agent

## 1. Type of agent

> **Answer: A goal-based problem-solving agent** (formulate goal → formulate problem → search → execute), with a utility-based component for choosing the cab type.

- **Goal-based:** the goal is clear – "reach the destination from the pickup location". The agent formulates the goal, defines the problem and searches for an action sequence (route + cab).
- **Utility-based component:** many cab types (Mini, Micro, Sedan, Prime, Shared…) can reach the goal; the best one is chosen by a utility function combining fare, waiting time (ETA), comfort and capacity according to the customer's preference.
- **Environment:** fully observable (map, fares, driver locations in the app), single customer agent, mostly deterministic, sequential, dynamic (traffic, availability) – so a search-based agent is appropriate.

## 2. Problem formulation

| Component | Definition |
|---|---|
| Initial state | Customer at pickup location, no cab booked |
| Actions | Enter destination; select cab type (Mini, Micro, Sedan, Shared, Prime…); confirm booking; go to pickup; drive along road segment to the next junction |
| Transition model | RESULT(state, action) → new state (e.g. cab booked → cab arrived → ride started → at destination) |
| Goal test | Customer is at the destination location |
| Path cost | Fare + travel time (+ waiting time); minimise (or maximise utility) |

## 3. Pseudocode

### (a) Simple problem-solving agent (general structure):

```
function OLA-CAB-AGENT(percept) returns an action
   persistent: seq   (an action sequence, initially empty)
               state (description of current world state)
               goal  (initially null)
               problem (a problem formulation)

   state <- UPDATE-STATE(state, percept)
   if seq is empty then
      goal    <- FORMULATE-GOAL(state)       # reach destination D from pickup P
      problem <- FORMULATE-PROBLEM(state, goal)
      seq     <- SEARCH(problem)             # e.g. UCS / A* on the road network
      if seq = failure then return NO-CAB-AVAILABLE
   action <- FIRST(seq)
   seq    <- REST(seq)
   return action
```

**(b) Booking procedure with cab-type selection:**

```
procedure BOOK-CAB(pickup, destination, preference)
  cabTypes <- {Mini, Micro, Sedan, Shared, Prime, ...}
  route    <- UNIFORM-COST-SEARCH(roadMap, pickup, destination)   # least time/cost path
  distance <- LENGTH(route)

  best <- null ; bestUtility <- -infinity
  for each cab in cabTypes do
    if AVAILABLE(cab, pickup) then
      fare <- COMPUTE-FARE(cab, distance, traffic)
      eta  <- ESTIMATE-ARRIVAL(cab, pickup)
      u    <- UTILITY(fare, eta, comfort(cab), capacity(cab), preference)
      if u > bestUtility then
        best <- cab ; bestUtility <- u
    end if
  end for

  if best = null then return "No cabs available – try again"
  display(best, fare, eta)
  if CUSTOMER-CONFIRMS(best) then
    driver <- ASSIGN-DRIVER(best, pickup)
    TRACK(driver, route)                  # execute the plan
    while not GOAL-TEST(currentLocation, destination) do
      update location ; reroute if traffic changes
    end while
    COMPLETE-TRIP() ; CHARGE(fare) ; GET-FEEDBACK()
  else
    let customer choose another cab type (go back to selection)
  end if
```

## CODE:

```
cabs = {
    # type: (fare per km, waiting time in min, comfort 1-10)
    "Micro":  (10, 4, 4),
    "Mini":   (12, 5, 5),
    "Shared": (7,  9, 3),
    "Sedan":  (16, 6, 7),
    "Prime":  (20, 7, 9),
}
def utility(fare, eta, comfort, pref):
    w_fare, w_eta, w_comfort = pref
    return -w_fare * fare - w_eta * eta + w_comfort * comfort

def book_cab(pickup, destination, distance_km, pref):
    print(f"Goal: travel from {pickup} to {destination} ({distance_km} km)")
    best, best_u = None, float("-inf")
    for name, (rate, eta, comfort) in cabs.items():
        fare = rate * distance_km
        u = utility(fare, eta, comfort, pref)
        print(f"  {name:7s} fare = Rs.{fare:6.1f}  wait = {eta} min  comfort = {comfort}  utility
= {u:7.2f}")
        if u > best_u:
            best, best_u, best_fare, best_eta = name, u, fare, eta
    print(f"\nSelected cab: {best} | Fare: Rs.{best_fare} | Arrives in {best_eta} min")
    for step in ["Booking confirmed", "Driver assigned", "Driver arrives at pickup",
                 "Ride started", f"Reached {destination}", "Payment done"]:
        print("  ->", step)

print("=== Budget customer ===")
book_cab("T. Nagar", "Airport", 15, pref=(1.0, 2.0, 5.0))
print("\n=== Comfort customer ===")
book_cab("T. Nagar", "Airport", 15, pref=(0.3, 1.0, 30.0))
```

## OUTPUT:

```
      Micro    fare = Rs. 150.0  wait = 4 min  comfort = 4  utility = -138.00
      Mini     fare = Rs. 180.0  wait = 5 min  comfort = 5  utility = -165.00
...   Shared   fare = Rs. 105.0  wait = 9 min  comfort = 3  utility = -108.00
      Sedan    fare = Rs. 240.0  wait = 6 min  comfort = 7  utility = -217.00
      Prime    fare = Rs. 300.0  wait = 7 min  comfort = 9  utility = -269.00

    Selected cab: Shared | Fare: Rs.105 | Arrives in 9 min
      -> Booking confirmed
      -> Driver assigned
      -> Driver arrives at pickup
      -> Ride started
      -> Reached Airport
      -> Payment done

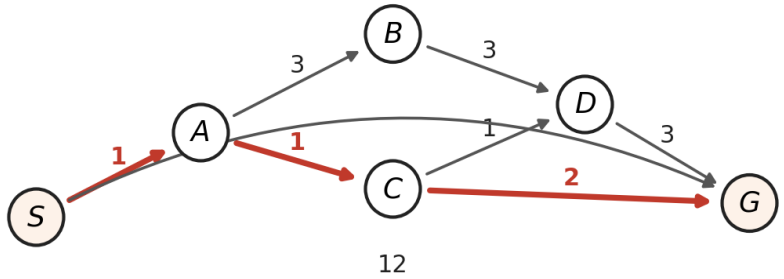
    === Comfort customer ===
    Goal: travel from T. Nagar to Airport (15 km)
      Micro    fare = Rs. 150.0  wait = 4 min  comfort = 4  utility =   71.00
      Mini     fare = Rs. 180.0  wait = 5 min  comfort = 5  utility =   91.00
      Shared   fare = Rs. 105.0  wait = 9 min  comfort = 3  utility =   49.50
      Sedan    fare = Rs. 240.0  wait = 6 min  comfort = 7  utility =  132.00
      Prime    fare = Rs. 300.0  wait = 7 min  comfort = 9  utility =  173.00

    Selected cab: Prime | Fare: Rs.300 | Arrives in 7 min
      -> Booking confirmed
      -> Driver assigned
      -> Driver arrives at pickup
      -> Ride started
      -> Reached Airport
      -> Payment done
```

## Question 5 – Uniform Cost Search (UCS) on the Delivery Network

**Problem:** Find the least-cost path from warehouse S (start) to warehouse G (goal) using UCS.

Delivery network (optimal path S-A-C-G highlighted, cost = 4)




### 1. Problem understanding

Directed weighted graph with edge costs (fuel + time):

| From | To (cost) |
|---|---|
| S | A (1), G (12) |
| A | B (3), C (1) |
| B | D (3) |
| C | D (1), G (2) |
| D | G (3) |
| G | goal |

### 2. Method selection and justification

UCS is chosen because edge costs differ. Unlike BFS, which minimises the number of edges, UCS always expands the node with the lowest path cost g(n) using a priority queue, so it is complete and optimal when all costs are positive. The goal test is applied when a node is expanded (removed from the queue), not when it is generated.

### 3. Algorithm

```
frontier <- priority queue ordered by g(n), containing S with g = 0
explored <- empty set
loop:
   if frontier is empty: return failure
   node <- POP(frontier)                # lowest g(n)
   if GOAL-TEST(node): return path to node
   add node to explored
   for each neighbour m of node with edge cost c:
      g(m) = g(node) + c
      if m not in explored and not in frontier: insert m
```

> else if m in frontier with higher g: replace it with the new cost

## 4. Step-by-step solution

| Iter. | Node expanded (g) | Frontier after expansion (node : g, via) | Explored |
|---|---|---|---|
| 1 | S (0) | A:1 (S), G:12 (S) | {S} |
| 2 | A (1) | C:2 (A), B:4 (A), G:12 (S) | {S, A} |
| 3 | C (2) | D:3 (C), B:4 (A), G:4 (C) ← G improved from 12 to 2+2 = 4 | {S, A, C} |
| 4 | D (3) | B:4 (A), G:4 (C) (D→G = 3+3 = 6 is worse, ignored) | {S, A, C, D} |
| 5 | B (4) | G:4 (C) (B→D = 4+3 = 7 is worse, ignored) | {S, A, C, D, B} |
| 6 | G (4) | Goal test passed → STOP | {S, A, C, D, B, G} |

**Order of expansion:** S → A → C → D → B → G.

**Path reconstruction (parent pointers):** G ← C ← A ← S, i.e. S → A → C → G.

**Cost:** S→A (1) + A→C (1) + C→G (2) = 4.

## 5. Comparison of candidate routes

| Route | Total cost | Remark |
|---|---|---|
| S → A → C → G | 4 | Optimal |
| S → A → C → D → G | 1 + 1 + 1 + 3 = 6 | Costlier |
| S → A → B → D → G | 1 + 3 + 3 + 3 = 10 | Costlier |
| S → G (direct) | 12 | Costliest (3× the optimal) |

## 6. Interpretation of result

> **Answer: Least-cost path: S → A → C → G with total cost = 4 cost units (fuel + time).**

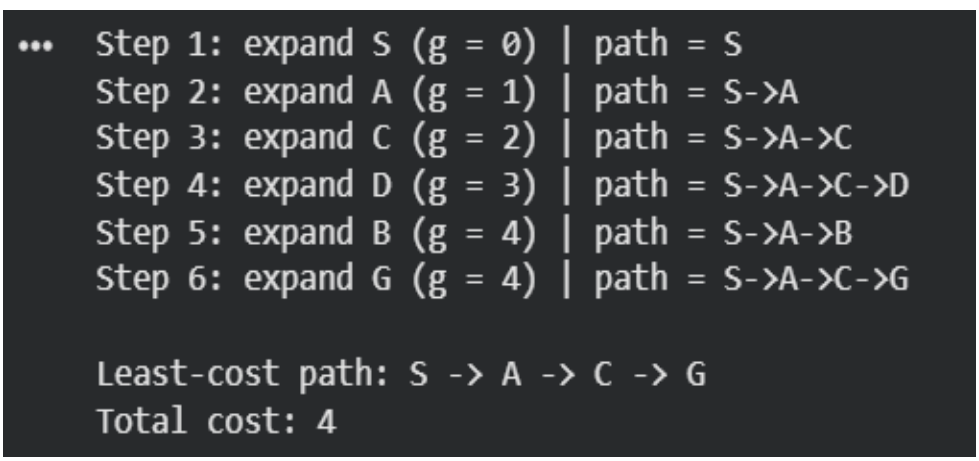
- The direct route S→G looks short (one hop) but is the most expensive (12); the multi-hop route through A and C costs only one-third as much.
- The company should dispatch the packages via warehouses A and C, saving 8 cost units (about 67%) compared with the direct route.
- UCS guarantees optimality because all edge costs are positive. Its time and space complexity is O(b^(1+⌊C*/ε⌋)) where C* is the optimal cost and ε the minimum edge cost.

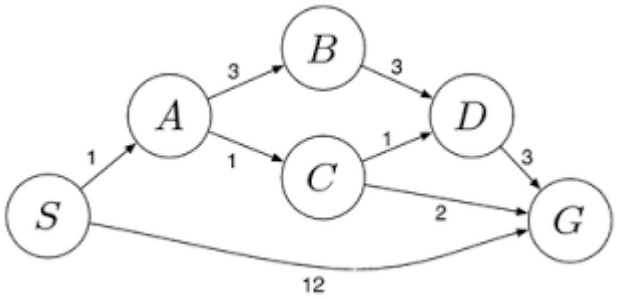
## CODE:

```
import heapq
graph = {
    "S": {"A": 1, "G": 12},
    "A": {"B": 3, "C": 1},
    "B": {"D": 3},
    "C": {"D": 1, "G": 2},
    "D": {"G": 3},
    "G": {},
}
def ucs(start, goal):
    frontier = [(0, start, [start])]       # (cost, node, path)
    explored = set()
    step = 1
    while frontier:
        cost, node, path = heapq.heappop(frontier)
        if node in explored:
            continue
        print(f"Step {step}: expand {node} (g = {cost}) | path = {'->'.join(path)}")
        step += 1
        if node == goal:
            return cost, path
        explored.add(node)
        for nbr, w in graph[node].items():
            if nbr not in explored:
                heapq.heappush(frontier, (cost + w, nbr, path + [nbr]))
    return None, None

cost, path = ucs("S", "G")
print("\nLeast-cost path:", " -> ".join(path))
print("Total cost:", cost)
```

## OUTPUT:






**RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Problem Understanding | 20% | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection | 20% | Selects most appropriate method/formula with strong justification | Selects correct method with minor errors | Inappropriate or partially correct method | Unable to select method |
| Solution Process | 25% | Logical, systematic steps with correct approach throughout | Mostly correct steps with minor mistakes | Disorganized steps; multiple errors | No clear process |
| Accuracy of Calculation | 20% | All calculations accurate; correct final answer | Minor calculation errors | Frequent errors; incorrect final answer | No valid calculations |
| Interpretation of Results | 15% | Clearly interprets results with units and engineering relevance | Basic interpretation with minor omissions | Limited or unclear interpretation | No interpretation |
| Problem Understanding | 20% | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps | Partial understanding; misses key elements | Misinterprets or unable to understand problem |